To generate the public and private key pair to use to sign the JWT you can use `keytool` from the command line, as follows. Keytool will generate a certificate for TLS when generating a public key pair, so use the `-dname` option to specify the subject name. This is required even though you won't use the certificate. You'll be prompted for the keystore password.

```
keytool -genkeypair \
  -keystore keystore.p12 \    #A
  -keyalg EC -keysize 256 -alias es256-key \    #B
  -dname 'cn=test'    #C
```

#A Specify the keystore
#B Use the EC algorithm and 256-bit key size
#C Specify a distinguished name for the certificate

> **TIP** Keytool picks an appropriate elliptic curve based on the key size, and in this case happens to pick the correct P-256 curve required for the `ES256` algorithm. There are other 256-bit elliptic curves that are incompatible. In Java 12 and later you can use the `-groupname secp256r1` argument to explicitly specify the correct curve. For `ES384` the group name is `secp384r1` and for `ES512` it is `secp521r1` (note: 521 not 512). Keytool can't generate EdDSA keys at this time.

You can then load the private key from the keystore in the same way that you loaded the HMAC and AES keys in chapters 5 and 6. Nimbus requires the private key to be cast to the more specific `java.security.interfaces.ECPrivateKey` type, so do that when loading the key: